Technical report — Klarbrief

- **Track:** Track 2B — Klarbrief: official Swiss letters in your language
- **Event:** Online
- **Team:** Klarbrief — Jannet Akanksha Ekka
- **Demo:** video https://github.com/JannetEkka/klarbrief/blob/main/track_2b/docs/klarbrief-demo.mp4 · code https://github.com/JannetEkka/klarbrief

1. Summary

People in Switzerland receive official letters (tax reminders, permit renewals, fines, premium subsidies) in German, French or Italian. 27% of residents do not hold a Swiss passport [1], and many newcomers cannot yet read the language of their commune well enough to catch a deadline. A language model can explain such a letter, but it can also state a deadline that is not in it, and the reader has no way to tell. **Klarbrief** has Apertus 1.5 explain the letter in the reader's language, as a short summary plus points: what to do, by when, how much, what happens otherwise, and who to contact. **Every point must quote the letter**, and a deterministic verifier finds that quote in the letter, checks that a deadline's date or an amount's value is written there, and rejects deadlines that predate the letter. Only points that pass are shown, with the letter's own words highlighted. The rest are listed apart, with the reason. On 48 replies from Apertus 1.5 8B, running offline on a 4-core CPU, Klarbrief showed **28 of 29 true deadlines, no wrong CHF amount, and 1 wrong date out of the 7 the model offered**, and every reply was in the reader's language. On 8 held-out letters written after the verifier was frozen, it showed **no wrong date or amount**.

2. Architecture

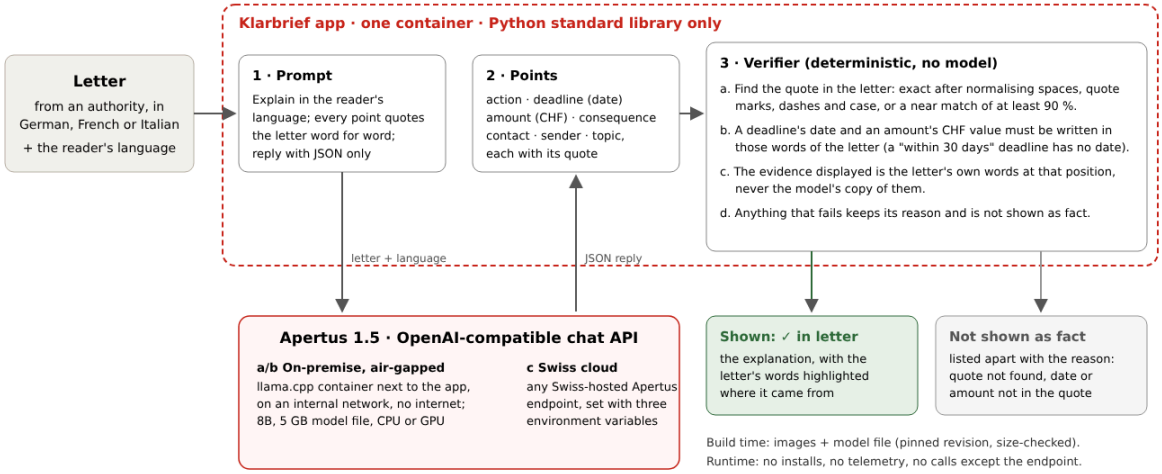


Figure 1. Klarbrief's pipeline. The verifier is ordinary code, not a model, so its decisions are reproducible and auditable.

1. **Prompt** (`src/klarbrief/explain.py`). One system prompt asks for a JSON object: a summary in the reader's language, and lists of actions, deadlines (ISO date or `null`), amounts (CHF), consequences and contacts, each with a `quote` copied from the letter. It also says that the letter's own date is not a deadline and that dates must not be calculated.
2. **Apertus 1.5** answers through the OpenAI-compatible chat API with `temperature 0` and the exact output schema (`response_format=json_schema`, which llama.cpp and vLLM enforce during generation). If an endpoint rejects it, the client steps down to plain JSON mode and then to text, says so, and records the mode in every reply.
3. **Verifier** (`src/klarbrief/verify.py`, about 250 lines, no model).
   - **(a) Find the quote.** It locates each quote after normalising whitespace, typographic quotes, dashes and case. It also accepts a near match of at least 90% similarity, or a partial match: a verbatim stretch covering at least 60% of the quote. For deadlines and amounts, any verbatim stretch of 12 or more characters that holds the same date or value also counts. It aligns to whole words and shows *the letter's* words at that position, never the model's copy.
   - **(b) Check the date.** A deadline's date must be written within the quoted words (±25 characters). A relative deadline ("innert 30 Tagen") must come with `date: null`. If the model gave no date but the matched words contain exactly one, that date is read from the letter.
   - **(c) Check the amount.** An amount's CHF value must be written within the quoted words.

    - **(d) Reject expired dates.** A deadline earlier than the letter's own dateline is rejected as already passed. Dates and amounts are parsed in DE/FR/IT/EN with Swiss number formats (`CHF 1'250.50`, `Fr. 85.–`).
4. **Language check** (`src/klarbrief/langcheck.py`). It identifies the letter's language and checks that the explanation is in the reader's language, using Unicode script for Ukrainian, Arabic, Persian, Tamil and Tigrinya and common words for Latin-script languages. A reply in the wrong language gets one retry ("rewrite in Ukrainian, keep quotes, dates and amounts"). If it is still wrong, the UI says so.
5. **UI** (`src/klarbrief/server.py`, `static/index.html`). One page served by the Python standard library. The reader pastes a letter, picks a language (15 offered, including Albanian, Tigrinya, Ukrainian, Tamil and Romansh), and sees cards. Hovering a card highlights its words in the letter. Rejected points sit in a collapsed "Not shown as fact" list with their reasons.

Target architecture (mandatory)

Klarbrief is deployable as **(a) on-premise** and **(b) air-gapped**, and is ready for **(c) a sovereign Swiss cloud**.

- **(a)/(b)** `make onprem` starts two containers. One is the official llama.cpp server with a 5 GB Apertus 1.5 8B GGUF. It sits on an `internal: true` Docker network with no route to the internet, and the model file is mounted read-only. The other is the app. The model file is the only build-time download (pinned revision, size-checked). At runtime nothing is installed or fetched, and nothing leaves the machine. Every result in this report was produced this way on a 4-core CPU with no GPU.
- **(c)** The app needs only `LLM_BASE_URL`, `LLM_API_KEY` and `LLM_NAME`, so it can be pointed at a Swiss-hosted Apertus endpoint. Infomaniak serves Apertus v1.5 70B through an OpenAI-compatible API [2], and Swisscom offers Apertus to business customers [3]. We have not run Klarbrief against those paid endpoints, so (c) is supported by the interface but untested.
- **Dependencies:** runtime is Python 3.12 standard library only (the app image installs nothing) plus llama.cpp's published image. Build time is Docker. Development only: `markdown` and Chromium to build this PDF.

3. Use of Apertus

- **Model:** Apertus 1.5 8B. On-premise we use a community text-only conversion of `swiss-ai/Apertus-v1.5-8B`, quantised to Q4_K_M GGUF (`Colby/apertus-v1.5-8b-text-Q4_K_M-GGUF@248ec68`, Apache-2.0). Track 2 allows community builds [4]. Image and audio input are not used.
- **How it is used:** inference only, one call per letter, `temperature 0`, `seed 0`, max 1,200 output tokens. No fine-tuning.
- **Where it runs:** llama.cpp server `--jinja` (the GGUF's embedded Apertus chat template), context 8,192, 4 parallel slots, 4 CPU threads.
- **Why Apertus:** its multilingual training (more than 1,000 languages [6]) covers the reader languages newcomers actually speak, and its open weights, data and licence let a commune run it on its own hardware. Both are what this use case needs.

4. Data

`data/letters/letters.json` holds **28 fictional official letters** written for this project. 20 are development letters (8 German, 6 French, 6 Italian). 8 are held-out letters (3 German, 3 French, 2 Italian), written after the verifier was frozen and used only for evaluation. They cover tax and fee reminders, permit renewals, parking fines, premium subsidies, a job-centre appointment, school registration and information-only notices. Names, communes ("Beispielwil", "Montexemple", "Esempiano"), addresses and case numbers are invented. No personal data is involved. Each letter is labelled with its true deadlines (calendar dates by which the reader must act) and every CHF amount it states. A unit test checks the labels against the text.

The letters deliberately contain traps a careless reader, human or model, falls for:

- the letter's own date;
- start dates ("ab dem 6. Januar 2027");
- expired dates ("am 31. März 2026 abgelaufen");
- the date of an offence;
- relative deadlines ("innert 30 Tagen", "dans les 30 jours", "entro 30 giorni"), which have no calendar date;
- three information-only letters.

Licence: CC-BY-4.0 (also offered under CDLA-Permissive-2.0). Every raw model reply is recorded in `data/results/*.jsonl`.

5. Evaluation

Task: explain each of the 20 dev letters in English and in Ukrainian (40 replies), and each held-out letter in English (8 replies). Then compare what would be shown with the labels.

Baseline: the same model's points shown *without* the verifier (blue in Figure 2), which is what a plain "explain this letter" app displays. **Ablations:** two earlier versions of the pipeline, kept in `data/results/` and re-scored with the final verifier.

Pipeline (Apertus 1.5 8B Q4_K_M, local CPU)	Letters · reader language	Replies	In reader's language	Points with quote found	True deadlines shown	Wrong deadline dates, raw → shown	Wrong CHF amounts, raw → shown	Invented deadline on a letter with none, raw → shown	Amounts visible	s / letter
Ablation A: plain JSON mode	20 dev · EN	20	20/20	79%	7/12	4 → 0	2 → 0	3 → 0 of 8	16/21	318
Ablation B: + output schema	20 dev · EN, UK	40	27/40	92%	24/24	4 → 2	0 → 0	4 → 2 of 16	40/42	301
Final: + reader language restated, checked	20 dev · EN, UK	40	**40/40**	92%	**24/24**	6 → 1	2 → **0**	6 → 1 of 16	37/42	300
Final, held out	8 held-out · EN	8	**8/8**	87%	4/5	1 → **0**	0 → 0	1 → **0** of 3	7/8	251

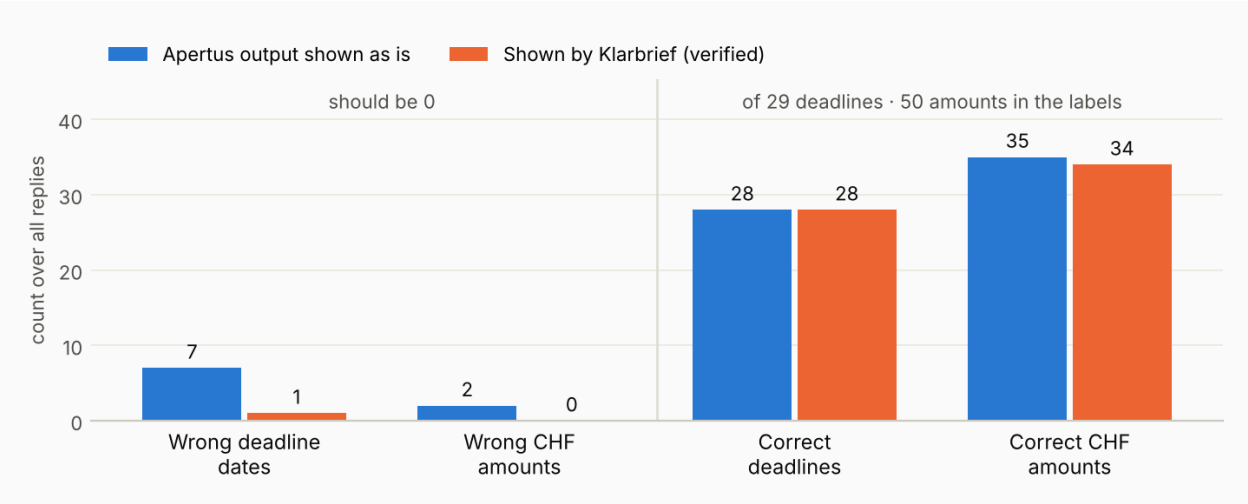


Figure 2. Final pipeline, all 48 replies: what a plain "explain this letter" app would show (blue) and what Klarbrief shows (orange). "Correct" counts true deadlines and CHF amounts listed as such. 44 of the 50 labelled amounts are visible to the reader somewhere in a verified point.

What the verifier caught.

- **Dates attached to relative deadlines.** Letters that say "innert 30 Tagen" or "entro 30 giorni" got the letter's own date (21 September, 19 October, 1 October) or the invoice date (14 October, held out). In an ablation run, a computed date (21 December) was attached too. All were blocked: the quoted words hold no such date, or the quote was not in the letter.
- **An expired date** (rule d, ablation run).
- **Two wrong CHF amounts** in each of ablation A and the final run.

What got through. The one wrong date shown in the final run is "ab dem 6. Januar 2027": the start of a new waste-collection day, presented as a deadline. The date really is in the letter, so only an interpretation check would catch it (§6).

Each engineering step was measured.

- **(A → B) Output schema.** With plain JSON mode, the quantised model sometimes produced valid JSON with a broken structure. All 5 true deadlines missing from run A came from replies with no `deadlines` field at all,

swallowed by a broken structure or skipped. Sending the exact output schema (`response_format=json_schema`; llama.cpp turns it into a grammar), in which every field is required, found all 24.

- **(B → Final) Reader language.** B exposed the next problem: only 7 of 20 Ukrainian explanations were actually in Ukrainian. The rest drifted into the letter's language or English, although the checked facts were the same. Restating the reader's language after the letter fixed all 20, at first try; the retry never had to fire.

Same facts in every language. For 19 of the 20 dev letters, the English and Ukrainian explanations showed the same deadlines and amounts. The facts are anchored in the letter's words, not in the reader's language.

Held out. The 8 held-out letters were written after the verifier was frozen. They cover new topics: electricity bill, military-service exemption tax, library reminder, childcare subsidy, census notice, vehicle inspection, school consent and vehicle tax. No wrong date or amount was shown. The one true deadline not shown (a vehicle-inspection appointment) was never offered by the model.

Cost. One letter takes about 790 prompt tokens and 440 output tokens. On the 4-vCPU machine (no GPU), with 4 parallel slots, the final 48-reply run took 60 minutes: about 48 letters an hour, at no marginal cost and with nothing leaving the machine. A commune can run this on hardware it already has. A GPU would make each reply interactive (not measured here).

6. Limitations

- **The verifier checks provenance, not interpretation.** A point whose quote and date are really in the letter but misread (a start date presented as a deadline, as in de-06) passes the checks. Rule (d) catches one such pattern, expired dates.
- **Tuned on the dev letters.** Rule (d), the partial-match rules, the "date read from the letter" rule and the reader-language reminder were added after inspecting replies to the 20 dev letters, so dev numbers are optimistic. That is why the 8 held-out letters exist. One held-out observation: the dateline parser does not recognise "Data fattura:" (ho-it-02), so rule (d) would not have applied there. We left the frozen verifier unchanged.
- **Strictness costs recall.** A correct point whose quote the model paraphrased beyond the near/partial rules is hidden. In the final run 30 of 323 points (9%) were hidden, mostly topics, contacts and actions whose quote was translated or paraphrased. Ukrainian replies lost slightly more. For deadlines and amounts the reader still saw 28 of 29 true deadlines and 44 of 50 amounts.
- **The explanation text itself is not verified.** The summary and each point's wording come from the model. The checks cover the quote, date and amount, not the translation. One English summary renamed the commune "Beispielwil" to "Examplewil". We did not have native speakers rate the Ukrainian output, and the language check detects the language, not its quality.
- **Small, synthetic test set.** 28 letters written by us (20 dev, 8 held-out), not real correspondence. Real letters bring scans, tables, payment slips (QR-bill) and legal boilerplate.
- **Quantised community build on CPU.** The numbers are for the Q4_K_M text-only 8B model at `temperature 0`, one run per letter, not the official BF16 checkpoint or the 70B. On a 4-core CPU one letter takes about 5 minutes; we did not measure GPU or hosted latency. llama.cpp's batched decoding is not bit-identical across batch sizes, so re-running can change individual replies slightly. Re-scoring the recorded replies is exact.
- **Not legal advice.** The UI points readers to the contact named in the letter.

7. Reproducibility

- `make test`: 34 unit tests (standard library `unittest`) in the app container. They include seam tests that run the whole explain → language check → retry → verify path against a stand-in model.
- `make run`: re-scores every recorded reply in `data/results/` (exact, no model needed), explains three letters live if an endpoint is configured, and serves the UI. Without an endpoint, the UI replays recorded replies for the sample letters, labelled as replayed.
- `make onprem`: downloads the pinned model and runs the whole stack offline. `make eval` regenerates `data/results/<model>.jsonl` (resumable).
- Hardware for the reported run: 4 vCPU, 16 GB RAM, no GPU; llama.cpp `ghcr.io/ggml-org/llama.cpp:server` (pulled 13 Oct 2026); `temperature 0`, `seed 0`, 4 parallel slots. Every reply carries its timestamp, token counts and output mode in `data/results/*.jsonl`. `make score` recomputes every number in this section exactly.

8. Next steps

- **Input:** photos and PDFs through Apertus 1.5's image input or local OCR, and the Swiss QR-bill (amount and due date come from its machine-readable code, a free cross-check).
- **Interpretation checks:** classify each date in the letter (letter date, start, expiry, deadline) with cheap rules and keep the model honest against them, closing the gap in Limitation 1.

- **Evaluation:** real, consented letters from a commune's integration office; native-speaker ratings of the explanations; the official BF16 8B and the 70B.
- **Pilot:** one commune's counter staff, on their existing hardware, where the "not shown as fact" list doubles as a review queue.

License

Code: Apache-2.0. This report, the docs and the letters: CC-BY-4.0. All Hack Apertus projects are open-sourced.

References

1. Avenir Suisse, "Nearly three in ten without a Swiss passport", based on FSO data, end of 2024. https://www.avenir-suisse.ch/en/blog-nearly-three-in-ten-without-a-swiss-passport/
2. Mastra model provider docs, Infomaniak (OpenAI-compatible API, `swiss-ai/Apertus-v1.5-70B`). https://mastra.ai/models/providers/infomaniak
3. Artificial Intelligence News, "Switzerland releases its own fully open AI model", Sept 2025. https://www.artificialintelligence-news.com/news/switzerland-releases-its-own-fully-open-ai-model/
4. Hack Apertus FAQ, "Is a community text-only GGUF of Apertus v1.5 acceptable?". https://hackapertus.ch/faq
5. Apertus team, "Apertus: Democratizing Open and Compliant LLMs", 2025. https://arxiv.org/abs/2509.14233
6. Swiss AI Initiative, Apertus (15T tokens, more than 1,000 languages). https://www.swiss-ai.org/apertus